

基于模型的强化学习 • Model-based RL

世界模型的两条路线

以及我把其中一条拆开验证之后

Dreamer (生成式 RSSM) • LeWorldModel (非生成式 JEPA)

分歧在哪

Dreamer

“能画出来，才算真懂。”

世界模型带 decoder，训练目标包含把潜在状态还原成像素。然后让策略完全在这个模型的想象里训练。

代价：为了预测球往哪飞，顺带得学会画背景里每一片草叶。

JEPA / LeWorldModel

“预测像素本身就是错的目标。”

完全没有 decoder。把观测编码成向量，只预测下一个向量。规划放到测试时做——CEM 搜索动作序列。

代价：没有任何东西阻止 encoder 塌缩成一个常数。必须额外加约束。

两者不共享优化目标（奖励 vs 与目标的距离），也不共享评估指标（累计回报 vs 成功率）。这一点后面还会回来。

我做了什么

1

手写 Dreamer

从零实现，单文件。RSSM、reward model、actor-critic、imagination rollout。

2

复现 LeWorldModel

跑通官方实现，复现出 86% 的规划成功率。

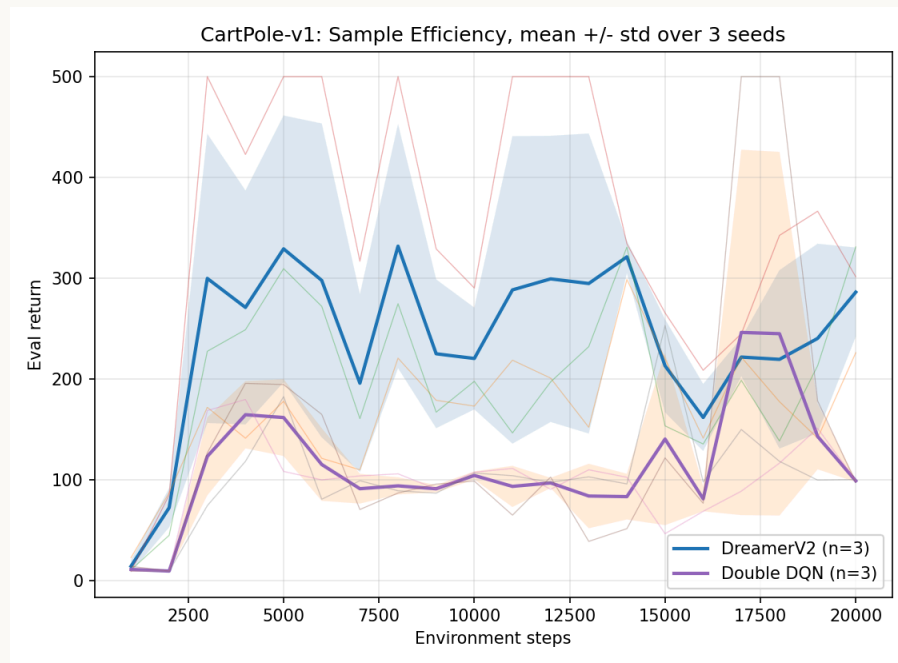
3

然后去验证它

三组受控实验。这才是值得讲的部分。

Dreamer，一行行写出来

约 670 行，不依赖任何框架。GRU 确定性状态 + 32×16 categorical 随机状态，straight-through 梯度，actor-critic 完全在想象轨迹上用 λ -return 训练。



240.1

Dreamer 平均回报

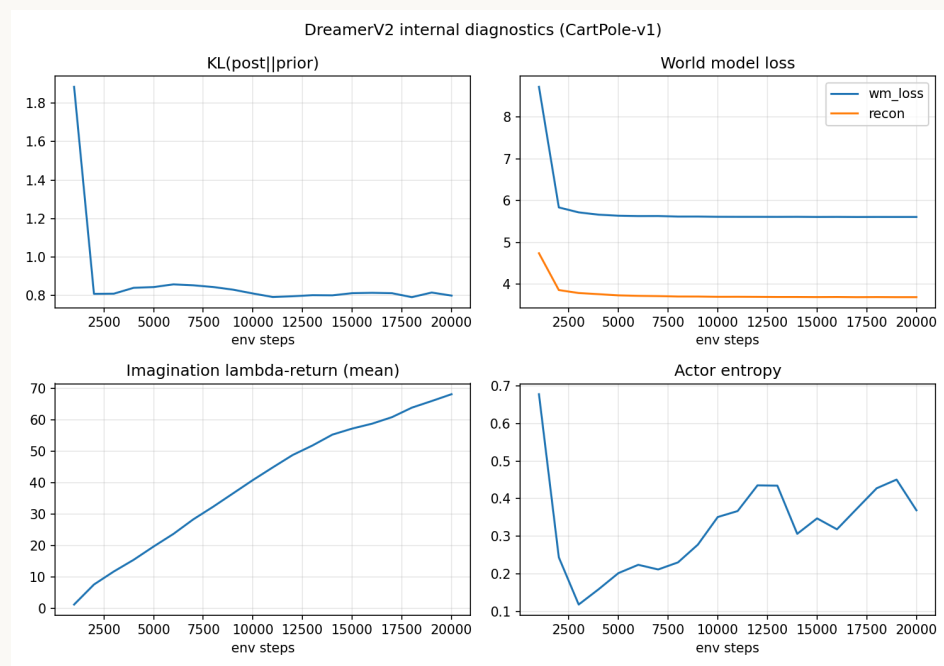
113.8

Double-DQN

CartPole-v1 · 2 万步 · 3 个 seed

想象出来的东西，对应现实吗？

actor 从头到尾没见过一个真实状态。所以关键问题是：想象轨迹到底跟不跟现实对齐， 还是说世界模型编了一个奖励很高但不存在的世界，策略在里面自我陶醉。



KL 稳定在 0.8 附近，free bits 正在起作用（既没后验塌缩也没爆炸）。想象中算出的 λ -return 从 0 涨到 70+，与真实评估回报同步——这就是"没有自欺"的证据。

LeWorldModel, 复现

86%

官方 checkpoint 的规划成功率

约 4 小时

花在装环境上，不是花在科研上

tworoom 任务，CEM-MPC，50 条留出 episode。这是后面所有数字的参照基准。

官方 README 和已发布的包之间有四处断裂：`box2d-py` 缺 `swig` 编译不过、`datasets` 解析到五年前的版本、`transformers` 5.x 重命名了所有 ViT 权重导致 checkpoint 加载失败、HDF5 支持被一个静默的 import 失败关掉了。这些我全写进了仓库——复现类文章通常略过这段，而这段恰恰是真正卡住人的地方。

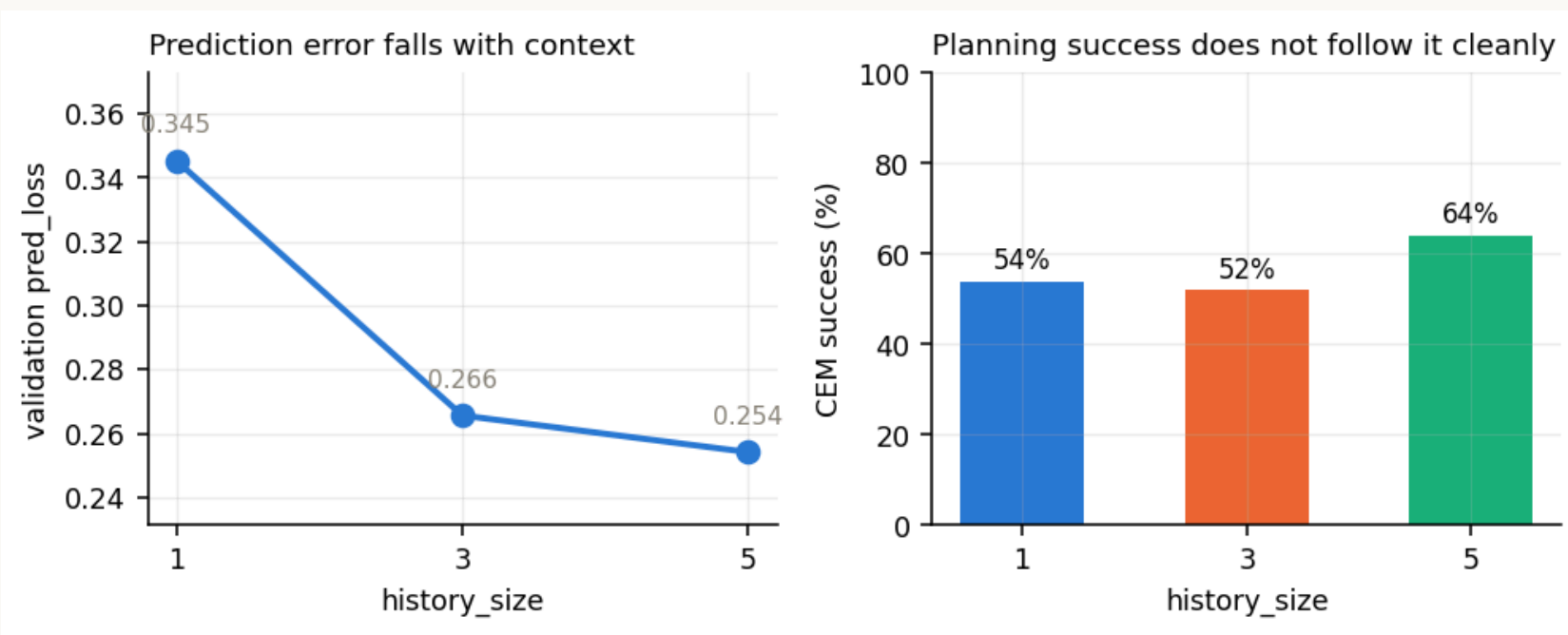
以上是复现

以下是我自己做的

同一份数据、同一个 encoder、同一个优化器、同样 900 步梯度更新、同样 50 条留出 episode。
每组实验只动一个变量。

实验一 —— 需要看多少历史？

`history_size` $\in \{1, 3, 5\}$, 即 predictor 能看到几帧过去的 embedding。3 是论文默认值。



loss 随上下文单调下降，成功率却没跟上。 $h=1 \rightarrow h=3$ 这一步，loss 改善了，成功率反而掉了 2 个点。50 条 episode 的标准误差约 7 个百分点，所以这一对就是噪声，我能下的结论只有 " $h=5$ 更好"。但正是这个错位，引出了后面两组实验。

实验二 —— 这一组我做错了

原计划：固定预算只换架构，把 PLDM 塞进同一个训练循环和规划器。跑出来 PLDM 预测更差却规划更好，看着像个漂亮的反例。

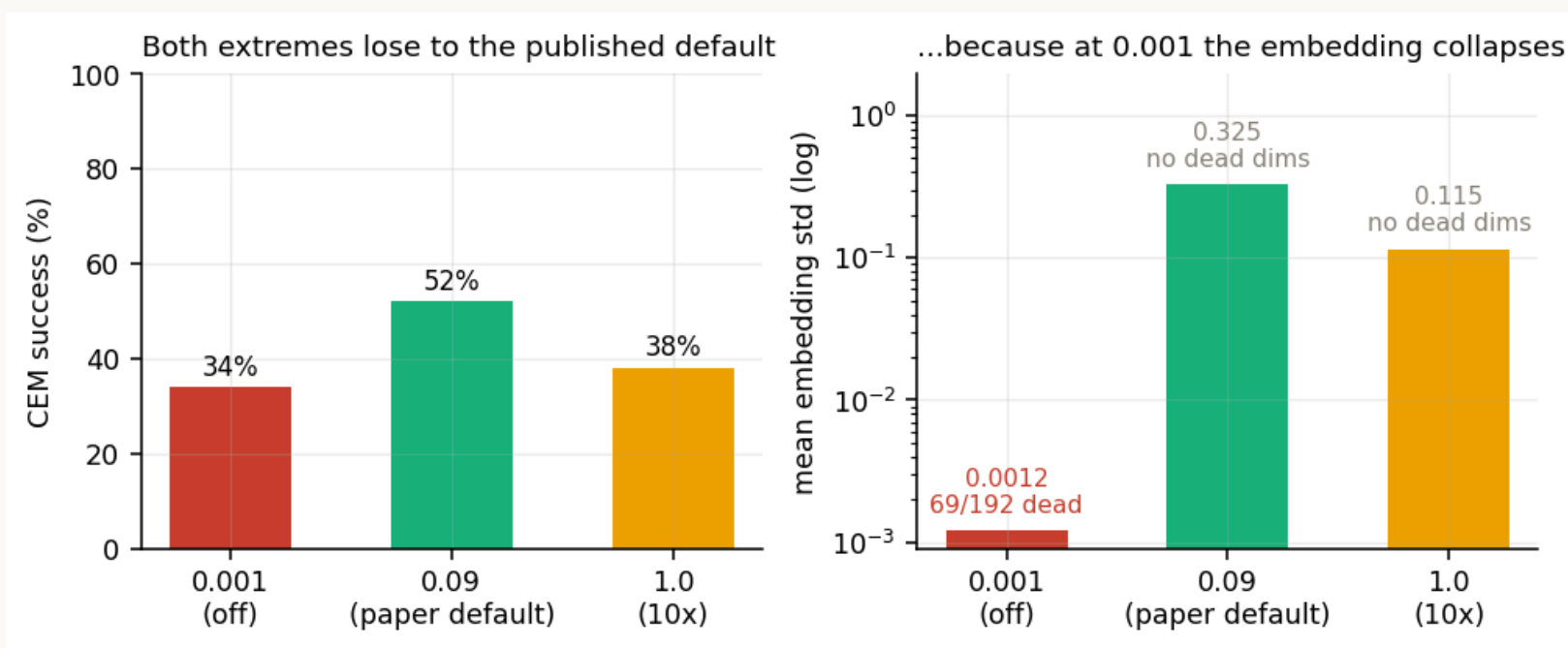
	pred_loss	规划成功率	死维度
LeWM h=3	0.266	52%	0 / 192
PLDM	0.298 更差	66% 更好	0 / 192

但它不成立。事后 diff 发现：pldm/module.py 和 lewm/module.py 逐字节相同；train.py 没给 Manager 传 seed，两次运行的初始权重不同（训练前 sanity loss 就是 5.147 vs 5.001）；14 个点 z=1.44、p=0.15、CI 跨过 0。

所以它实际是一次意外的重跑方差测量——同架构、同数据、同配方，仅初始化不同，成功率相差 14 个百分点。这给了整个项目一把尺子：**15 个百分点以内**的差距都不能当结论。

实验三 —— 于是我不再信 loss

LeWorldModel 的核心主张：两项 loss 就够，因为 SIGReg 会防止表征塌缩。我把它的权重扫到 0.001 和 1.0，并且没有去读 loss，而是直接测量 embedding 本身。



权重降到 0.001 时：pred_loss = 0.004，是我整晚训出来的最低值，比正常模型好 60 倍——同时 192 维里有 69 维已经死掉。encoder 塌缩会让预测任务变得极其简单，这个指标在这里是反向的。

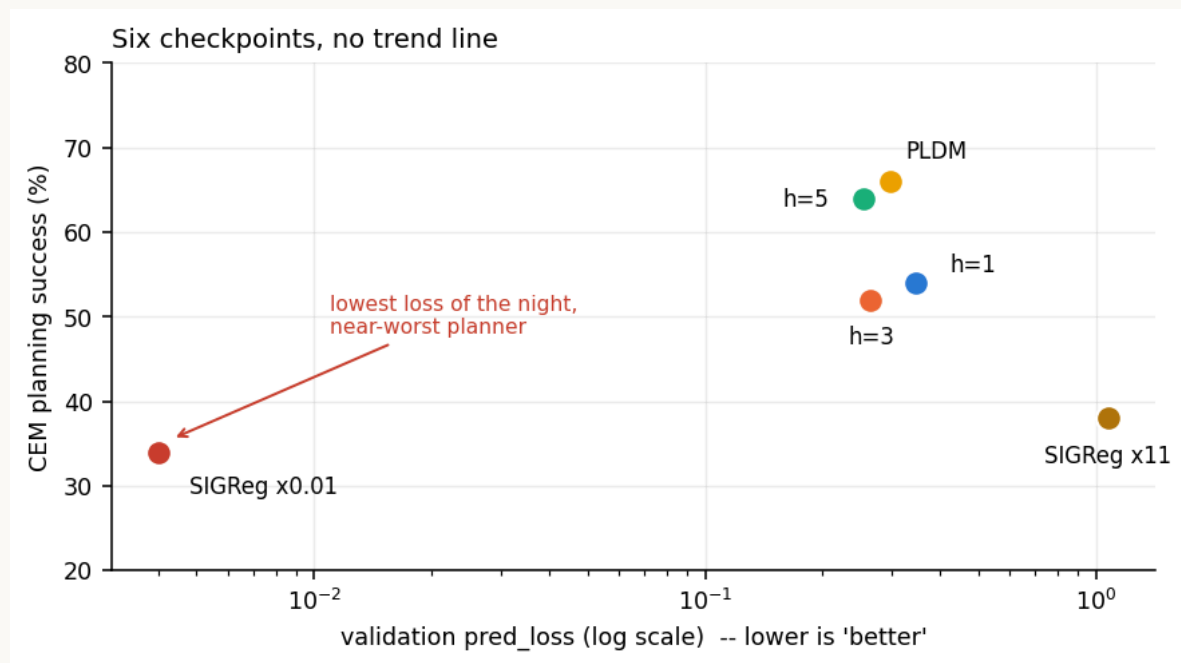
正则项没有失灵，它是被架空了

λ	pred_loss	sigreg_loss	emb 标准差	死维度	成功率
0.001	0.004	50.75	0.0012	69/192	34%
0.09 ★	0.266	5.16	0.325	0	52%
1.0	1.074	11.77	0.115	0	38%

SIGReg 一直在报警——它的值是 50.75，是健康状态的十倍。但权重只有 0.001，它对总 loss 的贡献只有 $0.001 \times 50.75 \approx 0.05$ ，而模型靠塌缩省下了 0.26 的预测误差。它出价出不过。

另一头是过度正则：不塌缩，但 embedding 被压得太紧（标准差 $0.325 \rightarrow 0.115$ ），区分状态的能力变弱。论文选的 0.09 正好落在这个倒 U 的顶点。

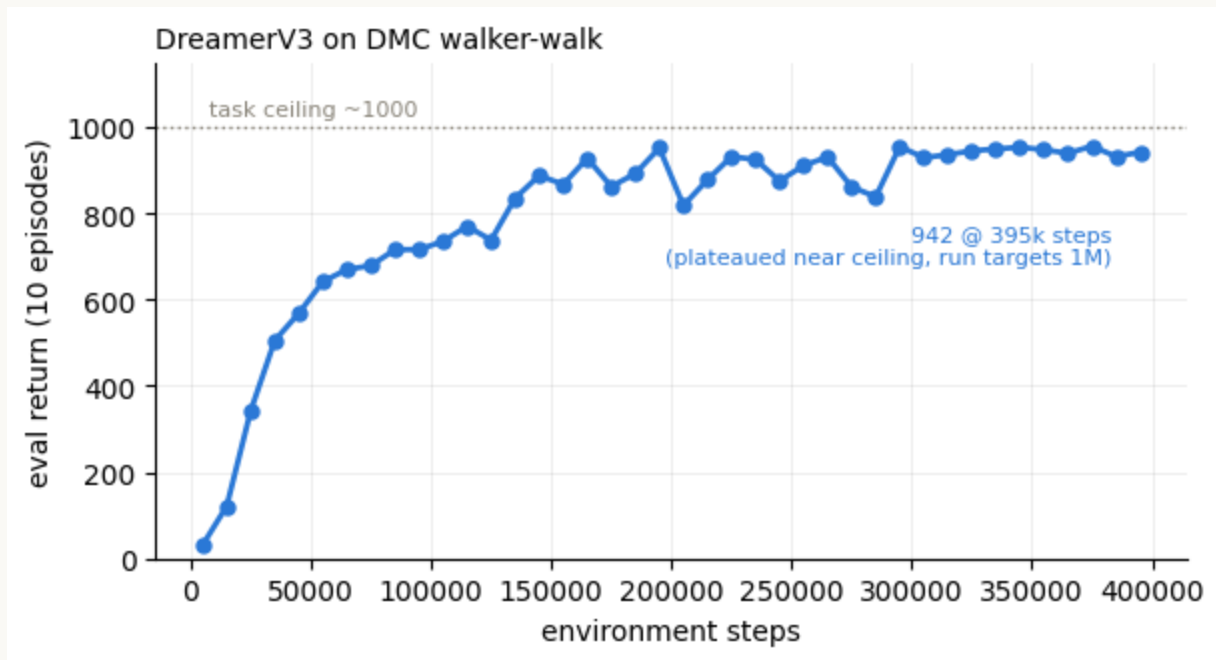
六个 checkpoint，没有趋势线



如果验证 loss 是可靠的代理指标，横跨三个数量级应该呈现一条向右下的趋势线。它没有。loss 最低的那个点是塌缩模型；规划最好的那个甚至不是 LeWM。

规模验证 —— DreamerV3 在 DMC walker-walk

手写版证明我理解每个组件；这一条是官方实现在真实基准上的完整曲线。



949

555k 步时的评估回报（满分约 1000）

曲线仍在上升，重新生成成本 deck 可刷新数值。

真正花掉时间的地方

把 **torch** 软链进别人的 **venv** 复用。uv 在处理一个没锁版本的 `torch>=2` 时顺着软链写穿了，悄悄把另一个项目的 torch 从 2.7 升到 2.14，而那个项目当时还有个跑了一天的进程在跑。

embed_dim 不是自由超参数。projector 的输入宽度取自它，而 ViT-tiny 的宽度固定在 192。我原计划的第一个消融直接崩在矩阵维度上——要正经扫它必须同步缩放骨干网络，那就不再是单变量消融了。

stdout 缓冲藏了两小时进度。终端日志卡在 45000 步一个多小时，而显式落盘的 `metrics.jsonl` 早就过了 80000 步。

完整记录在仓库的 engineering log 里。写下来是因为——出问题的地方，恰恰是没人愿意写下来的地方。

什么站得住，什么站不住

站得住

$\lambda=0.001$ 的塌缩：18 个点的成功率差距，并且有一个独立的机制测量指向同一方向。

官方 checkpoint 的 86%，对比我 900 步训练的所有模型。

站不住

单 seed、900 步、50 条评估。任何小于 10 个点的差距都是噪声——
h=1 vs h=3、PLDM vs h=5 都在此列。

没有 Dreamer 与 LeWorldModel 的正面对比：目标函数和评估指标都不共享，硬凑一根轴需要我没搭的共同任务定义。

下一步：先补到 3-5 个 seed 再谈其余结论；把 h 扫到 7-8 看收益在哪里饱和；以及把死维度数做成训练期的实时监控——它便宜，而且能抓到 loss 曲线藏起来的塌缩。